Design Patterns, GoF Mappings & Justifications

Pattern	Category	Design problem in TIVIDY	GoF participants	Collaboration	What varies	Why appropriate
1. Builder	Creational	A WorkflowDefinition is a deep tree of stages and tasks with dependencies, roles, deadlines, and routing tables. It cannot be built with a single constructor without one hundred parameters.	• **Builder**: WorkflowBuilder • **ConcreteBuilder**: StandardWorkflowBuilder • **Director**: WorkflowDirector (with subclasses DisputeDirector, ServiceActivationDirector, DocumentApprovalDirector) • **Product**: WorkflowDefinition	The Director's construct(builder) method issues beginStage / addTask / setRouting / setEscalationChain calls in the correct order. build() resets, calls construct, then getResult(), which validates and returns the product.	The workflow structure itself. Adding a new industry adds a new Director subclass; the builder and engine are unchanged.	Without a builder, WorkflowDefinition would need one constructor per workflow, or a giant static factory method. The pattern lets StandardWorkflowBuilder enforce invariants (open stages, validating dependencies) while each Director describes only the workflow.
2. Prototype	Creational	Each workflow instance needs its own deep copy of the definition's template tree — same structure, independent state. Editing one instance must never affect another or the definition.	• **Prototype**: WorkItem • **ConcretePrototype**: TaskItem, StageItem, WorkItemDecorator • **Client**: WorkflowDefinition::createInstance	createInstance calls template_root->clone(), injects instance data, and hands the clone to WorkflowInstance. StageItem::clone() deep-copies every child. SecureWorkItem::clone() clones the wrapped item and re-wraps it.	The structure of the template (which concrete task types, which decorators). createInstance never inspects it.	Factory Method would require a factory-per-task-type, and the factory would need to know every concrete class. Prototype lets the definition stay self-contained; createInstance treats the tree as opaque.

3. Abstract Factory	Creational	Each industry contributes a family of related products: a Director, participants, connectors, an assignment strategy, an escalation chain, and a UI profile. These must be created together, and adding an industry must not require editing the engine.	• **AbstractFactory**: IndustryPack • **ConcreteFactory**: BankingPack, TelecomPack, DemoPack • **AbstractProduct**: WorkflowDirector, Participant, ExternalConnector, AssignmentStrategy, EscalationHandler, ClientProfile • **Client**: Tividy::installPack()	installPack() calls each createX() in turn and registers the result. Each createX() has a working default, so a pack overrides only what differs.	The entire family of products per industry. Adding a third industry is one new class.	The pack is the only place where industry names appear. Tividy treats every pack identically via the abstract interface. This is what makes the "same engine, two industries" claim structurally true rather than aspirational.
4. Composite	Structural	A workflow contains nested work: a stage holds tasks and sub-stages. The engine must treat "a single task" and "a whole stage" uniformly for traversal, cancellation, visitor dispatch, and status queries.	Work-item tree • **Component**: WorkItem • **Leaf**: TaskItem • **Composite**: StageItem • Client: WorkflowInstance, FlowController, Tividy Command tree • **Component**: Command • **Composite**: MacroCommand	StageItem::status() derives from children. StageItem::cancel() cascades. StageItem::accept(visitor) visits itself then forwards. find(id) searches depth-first.	Tree shape per definition.	Without composite, FlowController would need if (item.isStage()) recurse; else process; at every traversal point. Composite makes stages and tasks indistinguishable to any caller that doesn't specifically need the distinction.
5. Decorator	Structural	A work item may need role-based permission checks, data validation, or other behaviours, but only some items, and in combinations.	• **Component**: WorkItem, Command • **ConcreteComponent**: TaskItem, StageItem	Each decorator holds its component by unique_ptr and forwards every method by default. SecureWorkItem::submit() checks the actor's role, then	Which concerns wrap which item, chosen per definition.	Permissions and validation are optional per item. Making them mandatory base-class concerns would burden every task; making them

		Subclassing gives 2^n combinations. Same for commands (retrying a command).	• **Decorator**: WorkItemDecorator, CommandDecorator • **ConcreteDecorator**: SecureWorkItem, ValidatedWorkItem, RetryingCommand	delegates. ValidatedWorkItem::complete() runs the rules, then delegates. RetryingCommand::execute() retries the wrapped command up to N times. Decorators stack: Secure(Validated(task)).		subclasses would explode combinatorially. Decorator keeps each concern in one class.
6. Adapter	Structural	Every external system (core banking, card scheme, credit bureau, network provisioning, SMS gateway) has its own API, and the engine must not depend on any of them.	• **Target**: ExternalConnector • **Adapter**: CoreBankingAdapter, CardSchemeAdapter, CreditBureauAdapter, NetworkProvisioningAdapter • **Adaptee**: CoreBankingVendor, CardSchemeVendor, CreditBureauVendor, NetworkVendor • **Client**: industry tasks like IssueProvisionalCreditTask, CreditCheckTask	A task builds a ConnectorRequest with an operation name and neutral params. invoke() translates to the vendor's own method call, catches its exceptions, and returns a ConnectorResult with success, retryable, message, payload. The task reads payload (e.g. ledger_ref, credit_score) and stores it in instance data.	Which vendor is behind each connector.	The engine knows nothing about vendor APIs. Swapping CoreBankingVendor for a real system is one #include change in the pack, and no engine class changes.
7. Façade	Structural	The TUI would otherwise orchestrate pack installation, definition lookup, instance creation, command execution, history	• **Façade**: Tividy • **Subsystem classes**: WorkflowLibrary, WorkflowDefinition, WorkflowInstance, ActionHistory,	The TUI calls only Tividy methods. startWorkflow internally calls library.get, definition.createInstance, instance.addObserver, instance.start. perform	The engine internals, but never the façade's public API.	The TUI stays 500 lines of screen logic instead of 2000 lines of engine orchestration. Every subsystem can evolve

		retrieval, deadline sweeping, and progress calculation across a dozen engine classes.	CommandFactory, ParticipantDirectory, ConnectorRegistry, WorkflowInstance, FlowController	internally calls commandFactory.create, actionHistory.execute.		independently as long as Tividy.h is stable.
8. State	Behavioural	A work item's lifecycle has 9 statuses and 10 transitions, and most operations are illegal in most statuses. Enforcing this with switch (status) would be the exact anti-pattern the spec forbids.	• **Context**: TaskItem • **State**: WorkItemState • **ConcreteState**: CreatedState, AvailableState, AssignedState, InProgressState, UnderReviewState, EscalatedState, CompletedState, CancelledState, SkippedState	TaskItem::publish() calls state->publish(*this). Each state overrides only the operations legal for it. CreatedState::publish() transitions to AvailableState and emits ItemAvailable. CompletedState overrides nothing, so every operation throws InvalidStateError via the default base implementation. States are stateless singletons (CreatedState::instance()).	Which transitions are legal, per status.	The state machine is the single largest single source of bugs in a workflow system. Isolating it into one class per state makes it locally testable (one test file per transition) and eliminates every switch/if on status across the engine.
9a. Strategy (routing ver.)	Behavioural	"What happens next when this item finishes?" varies per workflow and per source item. Banking routes on appeal outcomes; telecom routes on credit outcome, install type, and coverage result.	• **Context**: FlowController • **Strategy**: RoutingStrategy • **ConcreteStrategy**: DependencyRouting, OutcomeTableRouting, DataValueRouting	FlowController looks up the strategy by the completed item's id in DefinitionPolicies::routing. The strategy returns a RoutingDecision with activate and skip lists. FlowController publishes the activated items and skips the rest. DependencyRouting returns	Branching logic per workflow. Most industries use OutcomeTableRouting (data-driven table on Outcome::routingKey()) or DataValueRouting (table on a data field). No custom	A switch on outcome would be untestable, non-extensible, and would violate the spec's polymorphism rule. The strategy pattern lets each workflow declare its own routing as configuration.

				an empty decision (pure dependency release).	strategy classes are needed unless the branching rule is genuinely novel.	
9b. Strategy (assignment ver.)	Behavioural	"Who performs this work?" is a decision independent of routing. It varies by role, by data, and by attribute.	• **Context**: FlowController • **Strategy**: AssignmentStrategy • **ConcreteStrategy**: RoleBasedAssignment (default), DataDrivenAssignment (read a participant id from instance data), RegionalTechnicianAssignment (telecom-specific, filter technicians by region)	FlowController::assignIfPossible(item) calls assignment.chooseParticipant(item, requiredRole, directory, data). Empty return means nobody is eligible, the item stays Available.	Assignment policy per definition.	Two strategy families because routing and assignment vary independently. RoleBasedAssignment handles most cases; DataDrivenAssignment handles "this specific person, set at start"; RegionalTechnicianAssignment shows how a small custom strategy addresses a domain-specific rule without touching the engine.
10. Observer	Behavioural	Every state change must be recorded (audit), notified (email), and routed (next items). These are unrelated concerns that must not be coupled to WorkItem.	• **Subject**: EventSubject • **ConcreteSubject**: EventPublisher • **Observer**: EventObserver • **ConcreteObserver**: AuditObserver, NotificationObserver, FlowController	Every state transition publishes one WorkflowEvent. EventPublisher::notify() iterates over a copy of its observer list. AuditObserver appends to the audit log. NotificationObserver calls the "notifier" connector. FlowController reacts by	What reacts to what event.	New reactions (compliance reporting, analytics, SLA tracking) are new observer classes, not engine edits. Instance isolation is guaranteed by each instance having its own publisher.

				routing, assigning, and running automatic items. Each WorkflowInstance owns exactly one EventPublisher, so instances are isolated.		
11. Command	Behavioural	Every user action (start, submit, complete, approve, reject, escalate, cancel) must be undoable, loggable, describable, and compositeable (macros). Method calls can't be any of those.	• **Command**: Command • **ConcreteCommand**: StartCommand, SubmitCommand, CompleteCommand, ApproveCommand, RejectCommand, EscalateCommand, CancelCommand • **Receiver**: WorkItem • **Invoker**: ActionHistory • **Client**: CommandFactory	The TUI calls Tividy::perform(verb, ...). Tividy asks CommandFactory::create for the command, hands it to ActionHistory::execute. ActionHistory runs it, and pushes it on the undo stack on success.	Which actions exist (extensible via CommandFactory::registerVerb); how many composite into a macro.	Auditability requires every action be a first-class object. Undo requires that. MacroCommand (the Composite instantiation of Command) shows the pattern stacks with Composite cleanly.
12. Memento	Behavioural	Undo needs to restore an item's pre-action state, but the history must not see the item's private fields.	• **Originator**: TaskItem, WorkflowInstance • **Memento**: WorkItemMemento, InstanceMemento • **Caretaker**: InstanceCommand, ActionHistory (keeps the commands)	InstanceCommand::execute() captures instance.captureSnapshot() before calling doExecute. On failure, restores immediately. On undo, restores and publishes ItemReverted.	What each snapshot captures. Only the originator knows.	Without memento, undo requires either public state fields (breaks encapsulation) or a whole separate state-tracking system (breaks DRY). Memento gives the originator a private restore path.

13. Template Method	Behavioural	Every task must run in the same order: validate → work → record → transition → publish. Only the work itself varies per task.	• **AbstractClass**: TaskItem • **Primitive operation**: performWork(actor, outcome) (pure virtual) • **Hooks**: validateOutcome, onCompleted, performsIrreversibleWork • **ConcreteClass**: every industry task (CreditCheckTask, IssueProvisionalCreditTask, …) via CloneableTask<T>	TaskItem::executeWork runs the fixed sequence. If performWork throws ExternalFailure, the template catches it, publishes ItemFailed, and returns without transitioning. If it succeeds, onCompleted runs, then the calling state transitions the item and publishes ItemCompleted.	Only performWork's body.	Guarantees that every task publishes exactly one ItemCompleted event and records one outcome. Without the template, an industry task could forget to call onCompleted or skip the ItemFailed publication, breaking the audit trail.
14. Chain of Responsibility	Behavioural	Escalated items must reach a specific role depending on the escalation level, without the escalating item knowing which role that is.	• **Handler**: EscalationHandler • **ConcreteHandler**: RoleEscalationHandler	FlowController receives ItemEscalated and calls escalation_chain->handle({instanceId, itemId, reason, level, &data}). Each handler tests request.level == its_level. If matched, returns {resolved=true, approver_role, handled_by}. Otherwise forwards to successor. If the chain ends unresolved, FlowController publishes an EngineNote and leaves the item Escalated.	Which role handles which escalation level.	Banking escalates 1→2 for senior-analyst→compliance-officer; telecom escalates 1→2 for network-engineer→noc-manager. Both use RoleEscalationHandler with different configuration — no handler subclasses needed for these industries. A new escalation policy is a new handler class without touching FlowController.

15. Iterator	Behavioural	Several engine algorithms must walk every item in a tree (dependency release, deadline sweeping, progress counting), but the storage structure (std::vector<unique_ptr<WorkItem>>) must not leak into callers.	• **Iterator**: WorkItemIterator • **ConcreteIterator**: DepthFirstIterator • **Aggregate**: WorkItem::createIterator()	DepthFirstIterator(workflow_instance.root()) snapshots the traversal order at construction. FlowController::releaseEligible() uses it for pre-order traversal; DeadlineMonitor::sweep() uses it for deadline checks; ProgressVisitor uses it indirectly via accept.	Traversal order: one iterator per pattern. Additional iterators (AvailableWorkIterator, DependentsIterator) were considered and dropped as MVP cuts.	FlowController and DeadlineMonitor traverse the same tree with different intents. The iterator keeps the tree's storage a private detail.
16. Visitor	Behavioural	Reports over the work-item tree (progress, overdue list, compliance evidence) are open-ended, but the class hierarchy (TaskItem, StageItem) is stable.	• **Visitor**: WorkItemVisitor (visitTask, visitStage) • **ConcreteVisitor**: ProgressVisitor • **Element**: WorkItem::accept() • **ConcreteElement**: TaskItem::accept, StageItem::accept • **ObjectStructure**: WorkflowInstance::root()	Tividy::progress(instanceId) constructs a ProgressVisitor, calls instance.root().accept(visitor), and reads the accumulated counts. StageItem::accept calls visitor.visitStage(*this) once, then recurses into children.	The report. New reports are new visitor classes; no item class changes.	Without visitor, adding "count by status" would mean adding countByStatus() to WorkItem, then to TaskItem, then to StageItem, then to SecureWorkItem… every new report touches every class. Visitor keeps every report in its own class.